Mitigating signature churn: Backwards-compatible ladders in MTL-mode for Dynamic pq-DNSSEC

Minh Hoang Tran
tran189@vt.edu

Tijay Chung
tijay@vt.edu

Virginia Tech

Dynamic DNS requirements

Registry agreements require registries to publish zone changes within a set time.

The DNS update frequency defines an upper bound on the time between EPP confirmation and resolvable changes.

gTLD zone	Operator Registry	DNS update frequency (95% of probes)
base	-	60min
com.	Verisign	3 min
org.	Public Internet Registry	60 min
net.	Verisign	3 min
biz.	Registry Services (GoDaddy)	60 min
.info	Identity Digital	60 min

This means that within such a timespan, a registry must be able to:

- Put in zone file added/changed RRSet(s)
- Modify NSEC/NSEC3/NSEC5 appropriately
- **Sign RRSIGs for said RRSETs + NSEC**
- Update SOA counter, **sign SOA's RRSIG**
- Propagate change to all primary and secondaries

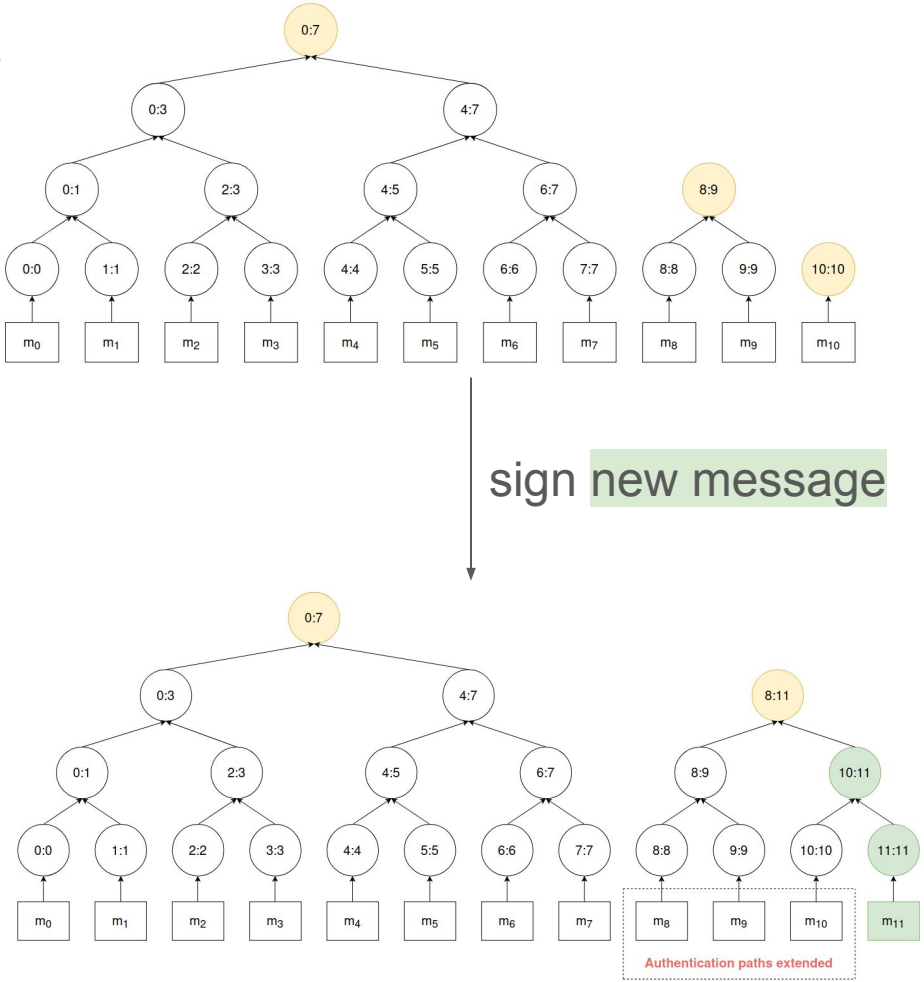
Under stateless DSAs, each signature is cryptographically independent from every other signature.

However, under the proposed MTL-mode, this is not necessarily the case.

Signature churn in MTL-mode

Under MTL-mode (stateful), condensed signatures are Merkle tree authentication paths:

- Signing new messages cause the authentication path of previous signed items to increase in length
- Necessitating the update/replacement of their corresponding signature entries



The condensed signatures that require updating may:

- be **variable in number** and
- correspond to RRSIGs completely **unrelated to the records added/changed** by DNS update

Quantifying consequence for Dynamic DNS

Average number of existing RRSIGs needing update per newly-added RRSIG:

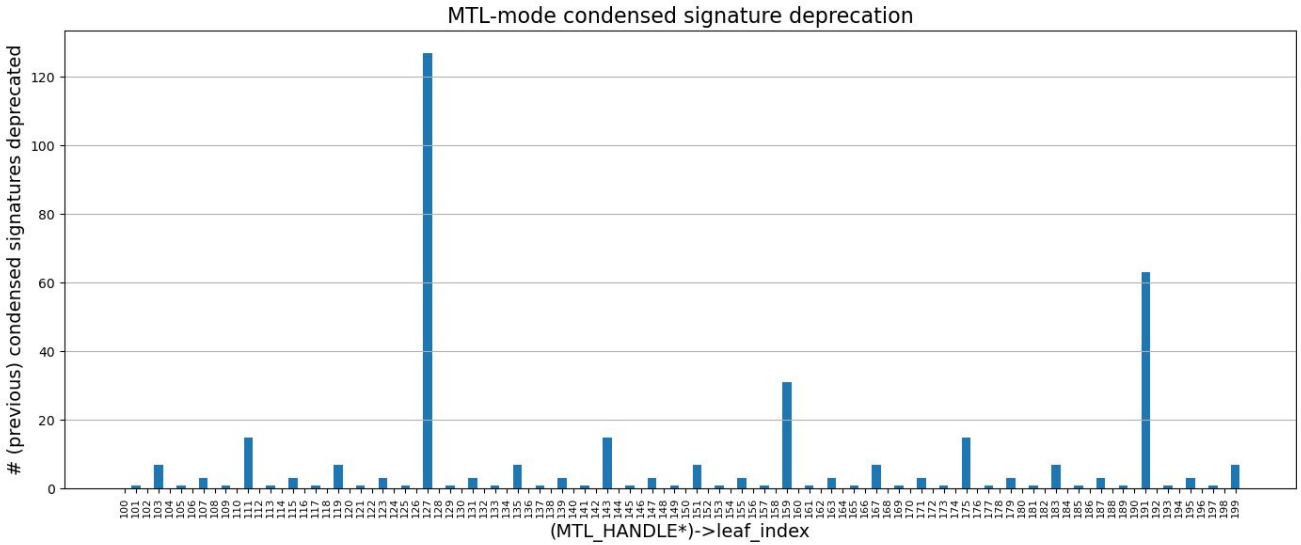
$$E[\text{updates}] = \frac{l-1}{2} = \frac{log_2(n)-1}{2}$$

- n: MTL node set size
  - (ie. # times key already used)
- l: bit length of n

	n	l	E[updates]
com.	≥ 13.6 M	≥ 24	≥ 11.5
org.	≥ 1.1 M	≥ 21	≥ 10

Actual number of updates required per new message signed:

- depends on the message handle index (+1)
- #updates: (greatest divisible power of 2) - 1

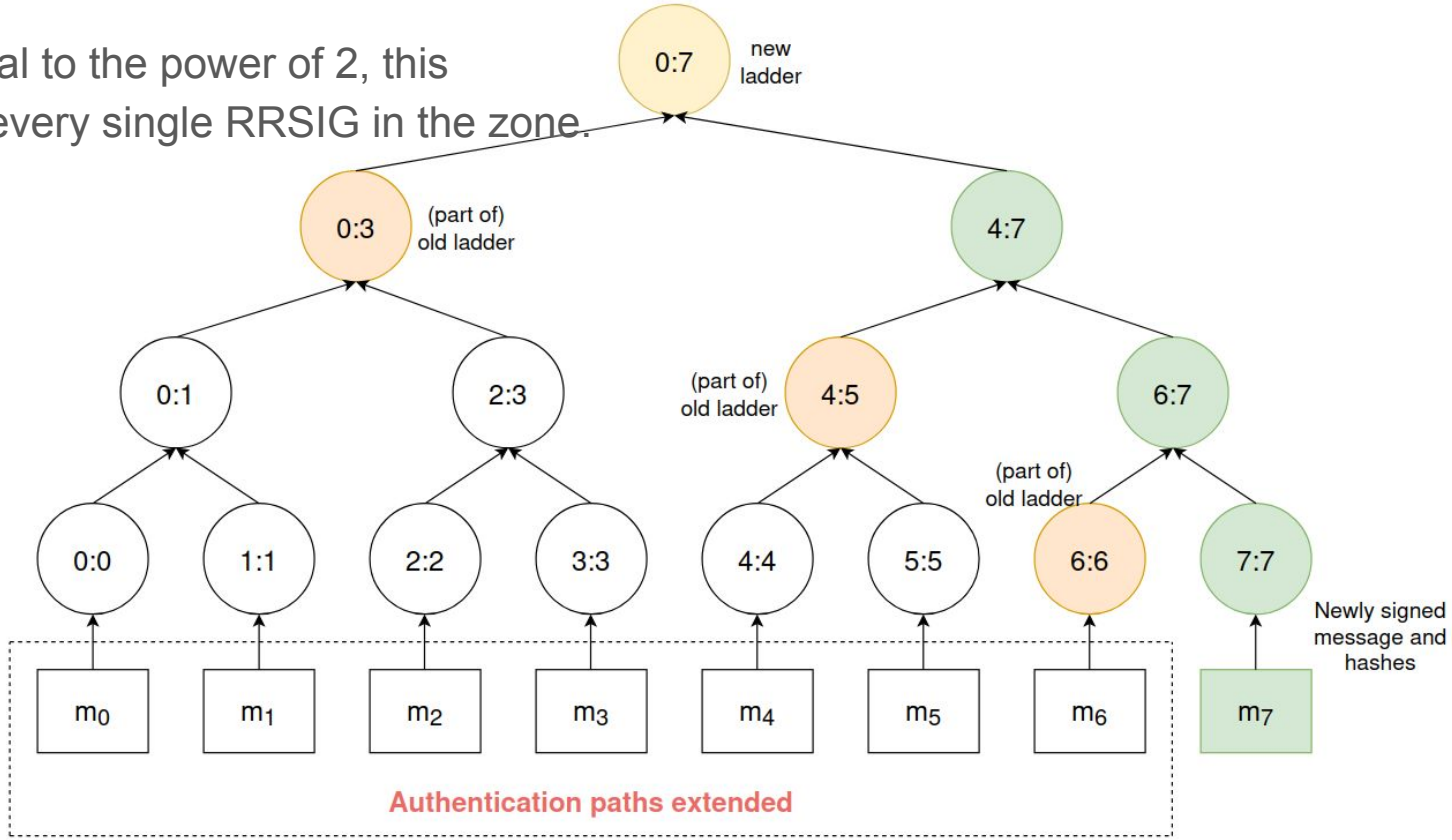


Signature churn worst case scenario

If index (+1) is equal to the power of 2, this involves updating every single RRSIG in the zone.

Additional note:

More recent signatures experience more frequent signature path churn than older signatures



Signature churn in MTLLIB (v1.2.0)

```
./mtl_test_dynamic

generating messages
generated messages

generating private key
generating corresponding public key
successfully generated keypair

signing A messages at time t1
obtaining ladder at time t1
successfully signed A at t1

signing B messages at time t2
obtaining ladder at time t2
exending A signatures (regenerating auth paths) at time t2
successfully signed B at t2

combinining ladders from t1 and t2
successfully created and signed backwards compatible ladder

verified both ladders from t1 and t2
verifying A_sigs_t1 against ladder_t1: 100/100 successes
verifying A_sigs_t2 against ladder_t2: 100/100 successes
verifying B_sigs_t2 against ladder_t2: 100/100 successes
```

```
ERROR (mtllib.c:mtllib_verify_with_ctx_str:1054): NULL mtl_rung
ERROR (mtllib.c:mtllib_verify_with_ctx_str:1054): NULL mtl_rung
ERROR (mtllib.c:mtllib_verify_with_ctx_str:1054): NULL mtl_rung
ERROR (mtllib.c:mtllib_verify_with_ctx_str:1054): NULL mtl_rung
ERROR (mtllib.c:mtllib_verify_with_ctx_str:1054): NULL mtl_rung
ERROR (mtllib.c:mtllib_verify_with_ctx_str:1054): NULL mtl_rung
ERROR (mtllib.c:mtllib_verify_with_ctx_str:1054): NULL mtl_rung
ERROR (mtllib.c:mtllib_verify_with_ctx_str:1054): NULL mtl_rung
ERROR (mtllib.c:mtllib_verify_with_ctx_str:1054): NULL mtl_rung
ERROR (mtllib.c:mtllib_verify_with_ctx_str:1054): NULL mtl_rung
ERROR (mtllib.c:mtllib_verify_with_ctx_str:1054): NULL mtl_rung
ERROR (mtllib.c:mtllib_verify_with_ctx_str:1054): NULL mtl_rung
ERROR (mtllib.c:mtllib_verify_with_ctx_str:1054): NULL mtl_rung
ERROR (mtllib.c:mtllib_verify_with_ctx_str:1054): NULL mtl_rung
ERROR (mtllib.c:mtllib_verify_with_ctx_str:1054): NULL mtl_rung
```

. . .

```
ERROR (mtllib.c:mtllib_verify_with_ctx_str:1054): NULL mtl_rung
ERROR (mtllib.c:mtllib_verify_with_ctx_str:1054): NULL mtl_rung
ERROR (mtllib.c:mtllib_verify_with_ctx_str:1054): NULL mtl_rung
ERROR (mtllib.c:mtllib_verify_with_ctx_str:1054): NULL mtl_rung
ERROR (mtllib.c:mtllib_verify_with_ctx_str:1054): NULL mtl_rung
ERROR (mtllib.c:mtllib_verify_with_ctx_str:1054): NULL mtl_rung
ERROR (mtllib.c:mtllib_verify_with_ctx_str:1054): NULL mtl_rung
ERROR (mtllib.c:mtllib_verify_with_ctx_str:1054): NULL mtl_rung
ERROR (mtllib.c:mtllib_verify_with_ctx_str:1054): NULL mtl_rung
verifying A_sigs_t1 against ladder_t2: 0/100 successes
```

Mitigations

Instead of immediately signing and propagating every EPP-induced zone changes, the follow strategies (not mutually exclusive) could be used to mitigate signature churn when MTL-mode is used on a dynamic zone:

(1) Batching:

- aggregate zone changes within a certain time span (ex: update freq / 3)
  - add all RRSETs' (hashes) to be signed to MTL node set
  - create and sign MTL-ladder **once**
  - propagate changes as a whole (instead of individually for each change)
- prevent rapid churn of a newly added (condensed) RRSIGs

(2) Backwards compatible MTL-ladders:

- signs (new) ladders that are compatible with old signatures (**existing RRSIGs need not be updated**)
- tradeoff: incur **slightly larger ladder size** (log-linear) to prevent signature churn
- also decouple need for synchronization between signed ladder (SOA's RRSIG) and condensed signatures (other RRSIGs)

Backward compatible MTL-ladder

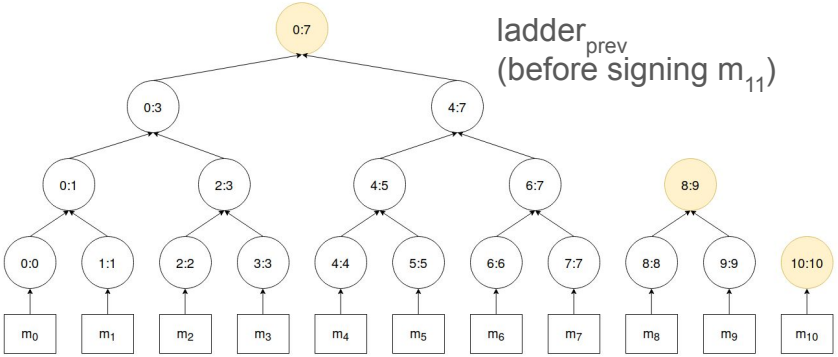
MTLLIB operation (MTLLIB v1.2.0):

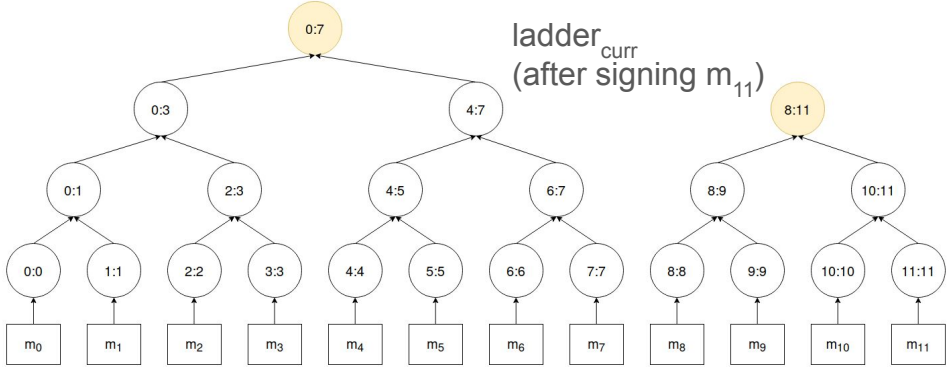
- Ladder represented as an array of rungs' (trusted roots') hashes
- mtl_verify() iterates through array, identifies **lowest rung** that brackets signature's leaf index
- mtl_verify() performs Merkle Tree verification (pairwise hashing) of condense signature (authentication path) against said rung

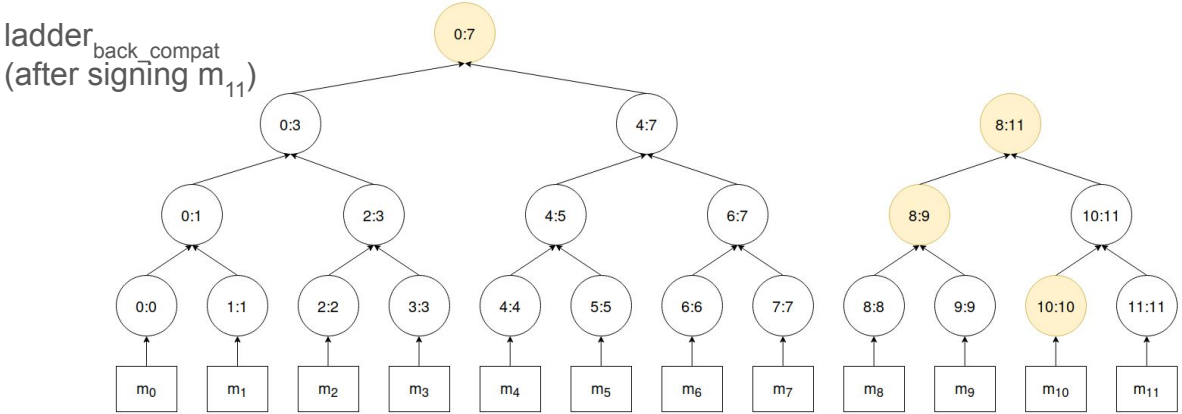
A backwards compatible can thus be implemented as a union between 2 ladders:

$$\text{ladder}_{\text{back_compat}} = \text{ladder}_{\text{curr}} \cup \text{ladder}_{\text{prev}}$$

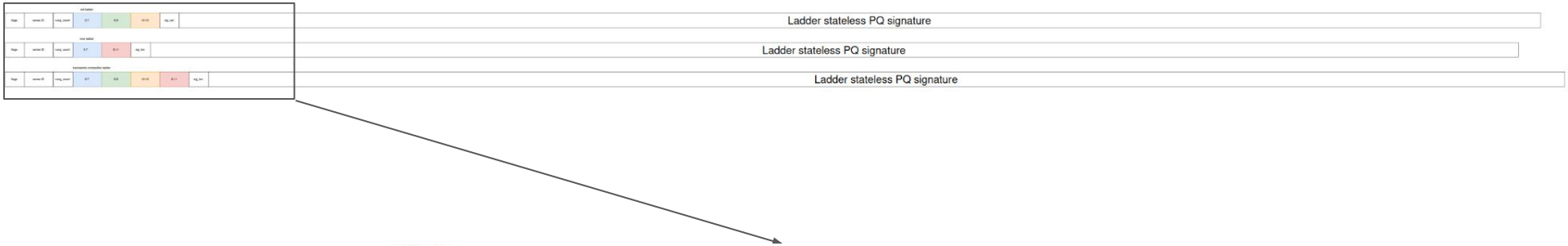
Backwards compatible MTL-ladder







Backwards compatible MTL-ladder (serialized)



old ladder

flags	series ID	rung_count	0:7	8:9	10:10	sig_len	pq_stateless_sig(flags, sid, rung_count, [(0:7), (8:9), (10:10)])

new ladder

flags	series ID	rung_count	0:7	8:11	sig_len	pq_stateless_sig(flags, sid, rung_count, [(0:7), (8:11)])

backwards compatible ladder

flags	series ID	rung_count	0:7	8:9	10:10	8:11	sig_len	pq_stateless_sig(flags, sid, rung_count, [(0:7), (8:9), (10:10), (8:11)])

BC-ladder size

Size growth of backwards compatible ladder after batch signing k-messages:

$$\begin{aligned}\text{len}_{\text{BC_ladder}} &= \text{len}_{\text{prev_ladder}} + E[\text{new_rungs}] \\ &= \text{len}_{\text{prev_ladder}} + \frac{\lfloor log_2 k \rfloor + 1}{2} + \frac{k}{2^{\lfloor log_2 k \rfloor + 1}} \\ &\approx \text{len}_{\text{prev_ladder}} + \frac{log_2 k}{2} + 1\end{aligned}$$

- logarithmic to the number of messages per batch
- linear to the number of times backward-compatible consecutively used

Backwards compatible ladders should thus:

- not be used for every single zone updates (batched or otherwise)
- but rather when use of a normal MTL ladder would require too many condensed signatures to be extended/updated to be feasible

Proof of concept implementation

```
./mtl_test_dynamic

generating messages
generated messages

generating private key
generating corresponding public key
successfully generated keypair

signing A messages at time t1
obtaining ladder at time t1
successfully signed A at t1

signing B messages at time t2
obtaining ladder at time t2
exending A signatures (regenerating auth paths) at time t2
successfully signed B at t2

combinining ladders from t1 and t2
successfully created and signed backwards compatible ladder

verified back_compat_ladder
verifying A_sigs_t1 against back_compat_ladder: 100/100 successes
verifying A_sigs_t2 against back_compat_ladder: 100/100 successes
verifying B_sigs_t2 against back_compat_ladder: 100/100 successes
```

Discussion

Additional advantages

- Relaxes synchronization need between condensed signatures (RRSIGs) and signed ladders (SOA's RRSIG)
- Increased operational flexibility
- Additional zone changes from being delayed while condensed signatures are updated and propagated

Next steps:

- Experimental simulation
- DNS transport improvements (1-RTT DoTCP, UDP-based streams, etc)